



UNIVERSIDAD ESTATAL AMAZÓNICA

Unidad de Organización Curricular:

Campo de Estudio

INFORME DE PRÁCTICAS DEL COMPONENTE PRÁCTICO-EXPERIMENTAL	
Nombre(s) del(de los) Estudiante(s): MINAYA GARCIA JEFFERSON CRISTOPHER GABRIEL ARIEL VELASCO MOLINA	
Asignatura: ESTRUCTURA DE DATOS "E"	Calificación:
Fecha del informe: 1/09/2026	
Nº Práctica: 3	
Título de la Práctica: Implementación de conjuntos y mapas.	
Introducción	La presente práctica experimental aborda la aplicación de estructuras de datos en una solución tecnológica para la gestión básica de una biblioteca. Se seleccionó el programa de registro de libros en una biblioteca porque constituye un escenario concreto en el que se deben identificar registros de forma única, evitar duplicados, consultar información y controlar la disponibilidad de recursos. El objetivo es desarrollar una aplicación funcional en Python que utilice conjuntos, mapas y diccionarios para registrar, organizar y consultar un catálogo digital. En Python, un diccionario implementa una relación entre claves y valores: una clave, como un ISBN, permite recuperar el registro asociado de manera directa [1]. Desde la perspectiva abstracta, un mapa representa una correspondencia entre un conjunto de claves y un conjunto de valores; sus operaciones principales son insertar, consultar, actualizar y eliminar pares clave-valor [2]. Los conjuntos, por su parte, representan colecciones sin duplicados y son adecuados para operaciones de pertenencia e identificación de elementos únicos. La implementación de estas estructuras se relaciona con las tablas hash. Tapia-Fernández, García-García y García-Hernández explican que las tablas hash ofrecen complejidad amortizada constante en varias operaciones, aunque su rendimiento real depende de factores como

	las colisiones, el lenguaje y la implementación [2]. Esta consideración justifica la inclusión de una medición empírica, además del análisis teórico. La práctica se justifica porque una biblioteca requiere búsquedas frecuentes y consistencia en sus identificadores. Un mapa ISBN–libro evita recorrer el catálogo completo para encontrar un ejemplar, mientras que los conjuntos permiten gestionar autores únicos, libros disponibles y los títulos asignados a cada usuario. La solución es deliberadamente pequeña y de consola, pero reproduce operaciones presentes en sistemas de información más grandes y permite observar con claridad las ventajas y limitaciones de cada estructura.																		
Desarrollo o procedimiento	<i>Descripción de la actividad y escenario</i> La actividad se desarrolló como una práctica de programación en un entorno de escritorio virtual/local, utilizando Python 3.11 y un editor de texto. El escenario corresponde a un laboratorio de desarrollo de software: se trabajó con archivos de código, terminal, pruebas automatizadas y generación de evidencias. No fue necesario utilizar espacios abiertos ni equipamiento especializado; el sistema funciona sin conexión a una base de datos externa y no emplea datos personales reales. El procedimiento consistió en definir el problema, modelar las entidades del catálogo, seleccionar la estructura más conveniente para cada relación, implementar las operaciones, ejecutar casos de prueba, generar reportes y comparar tiempos de acceso. Los registros de demostración son libros de tecnología y computación; se emplean únicamente con fines académicos. <i>Diseño de la solución</i> <table><tr><th>Estructura</th><th>Variable</th><th>Función dentro del sistema</th></tr><tr><td>Diccionario</td><td>libros</td><td>Relaciona cada ISBN con otro diccionario que contiene título, autor, categoría y año.</td></tr><tr><td>Conjunto</td><td>disponibles</td><td>Mantiene los ISBN que pueden prestarse, sin duplicados.</td></tr><tr><td>Conjunto</td><td>autores</td><td>Registra una sola vez cada autor del catálogo.</td></tr><tr><td>Mapa</td><td>libros_por_autor</td><td>Relaciona un autor con un conjunto de ISBN.</td></tr><tr><td>Mapa</td><td>prestamos</td><td>Relaciona un usuario con el conjunto de ISBN que tiene prestados.</td></tr></table>	Estructura	Variable	Función dentro del sistema	Diccionario	libros	Relaciona cada ISBN con otro diccionario que contiene título, autor, categoría y año.	Conjunto	disponibles	Mantiene los ISBN que pueden prestarse, sin duplicados.	Conjunto	autores	Registra una sola vez cada autor del catálogo.	Mapa	libros_por_autor	Relaciona un autor con un conjunto de ISBN.	Mapa	prestamos	Relaciona un usuario con el conjunto de ISBN que tiene prestados.
Estructura	Variable	Función dentro del sistema																	
Diccionario	libros	Relaciona cada ISBN con otro diccionario que contiene título, autor, categoría y año.																	
Conjunto	disponibles	Mantiene los ISBN que pueden prestarse, sin duplicados.																	
Conjunto	autores	Registra una sola vez cada autor del catálogo.																	
Mapa	libros_por_autor	Relaciona un autor con un conjunto de ISBN.																	
Mapa	prestamos	Relaciona un usuario con el conjunto de ISBN que tiene prestados.																	

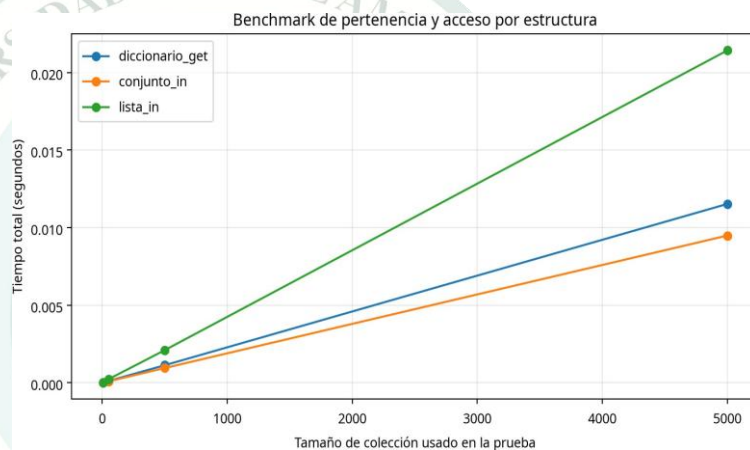
	El ISBN funciona como clave porque identifica de manera práctica cada registro del catálogo. El valor asociado es un diccionario de atributos, lo que permite agrupar información heterogénea sin crear una clase adicional. El conjunto disponibles se actualiza al prestar y devolver un libro: la operación de eliminación retira el ISBN y la operación de adición lo reincorpora. El mapa <code>libros_por_autor</code> combina un mapa con conjuntos para representar una relación de uno a muchos y evitar repeticiones. <i>Procedimientos implementados</i> La aplicación incluye un menú de consola con siete opciones: agregar libro, buscar por ISBN, buscar por autor, registrar préstamo, registrar devolución, mostrar reporte y ejecutar una demostración con benchmark. La función <code>agregar_libro</code> valida que el ISBN no exista, crea el diccionario de atributos, incorpora el ISBN al conjunto de disponibles y actualiza los índices de autor. Las funciones <code>prestar</code> y <code>devolver</code> verifican la disponibilidad y la pertenencia del ISBN al conjunto del usuario antes de modificar los mapas. El reporte general presenta el total de libros, disponibles, prestados y autores únicos. Además, muestra el catálogo, el mapa autor–ISBN y el mapa usuario–préstamos. De esta manera se cumple el requisito de visualizar y consultar los elementos que conforman las estructuras de datos. <i>Agente de IA utilizado</i> Se utilizó un agente de inteligencia artificial generativa como apoyo para proponer la organización inicial del código, revisar la correspondencia entre el problema y las estructuras, sugerir pruebas y estructurar la redacción técnica. La participación humana comprende la selección del tema, revisión de requisitos, adaptación del código, ejecución, interpretación de resultados y verificación de las salidas. De forma transparente, se estima que aproximadamente el 70 % del código recibió asistencia del agente y el 30 % correspondió a revisión, adaptación y validación humana. Esta proporción debe ajustarse si el estudiante modifica el código o si su participación real fue diferente.
Resultados	<i>Funcionamiento observado</i> La demostración se ejecutó con cinco libros y dos préstamos. El reporte mostró cinco libros registrados, tres disponibles, dos prestados y cinco autores únicos. La consulta por el autor Eric Matthes devolvió el ISBN 978-1492051367 y el título Python Crash Course. El mapa de usuarios presentó los préstamos de Ana Torres y Luis Gómez. La salida completa se conserva en <code>evidencias/salida_demo.txt</code> y se incluye una captura en

los anexos.

Las pruebas automatizadas verificaron cuatro comportamientos: inicialización de conjuntos y mapas, rechazo de ISBN duplicado, ciclo préstamo–devolución y búsqueda por autor. El resultado fue Ran 4 tests ... OK, por lo que los casos definidos finalizaron correctamente.

Análisis del tiempo de ejecución

El programa incluye un benchmark reproducible con timeit. Se midieron accesos a un diccionario, pertenencia en un conjunto y pertenencia en una lista, usando 5, 50, 500 y 5000 como tamaños de colección de referencia. En una corrida representativa, para 5000 elementos y 500000 operaciones, el diccionario tardó aproximadamente 0.0115 s, el conjunto 0.0095 s y la lista 0.0214 s. Los valores pueden variar según el procesador y la carga del sistema; por esa razón se conserva el archivo CSV con las mediciones reales.



1 Comparación de tiempos medida con el script benchmark.py.

Estructura	Operación	Complejidad promedio esperada	Ventaja y limitación
Diccionario	Acceso por clave	$O(1)$ amortizado	Consulta directa; requiere claves ordena por naturaleza.
Conjunto	Pertenencia, alta y baja	$O(1)$ amortizado	Evita duplicados; indexable.
Lista	Pertenencia	$O(n)$	Simple y ordenada; el costo crece al recorrer elementos.

El resultado experimental coincide con la expectativa teórica: el acceso

	por clave y la pertenencia en un conjunto se mantienen rápidos, mientras que la búsqueda lineal en una lista aumenta más conforme crece la colección. Sin embargo, la notación $O(1)$ es amortizada y no significa que todas las operaciones duren exactamente lo mismo. Las colisiones, la memoria disponible y la implementación influyen en el tiempo observado [2]. <i>Ventajas y desventajas en el problema</i> Los conjuntos son ventajosos porque garantizan unicidad y expresan con claridad operaciones como “está disponible” o “este autor ya existe”. Su principal desventaja es que no ofrecen acceso por posición ni un orden de presentación inherente; por ello el reporte ordena los elementos antes de mostrarlos. Los mapas y diccionarios permiten búsquedas eficientes y modelan relaciones naturales como ISBN–libro, autor–ISBN y usuario–préstamos. Como desventajas, consumen memoria adicional para mantener la tabla hash y exigen claves adecuadas; además, un diccionario en memoria no garantiza persistencia cuando termina el programa.
Conclusiones	La práctica alcanzó el objetivo de implementar una aplicación tecnológica funcional para registrar y consultar libros mediante conjuntos, mapas y diccionarios. El diccionario principal permitió localizar cada libro por ISBN, los conjuntos resolvieron la unicidad de autores y el control de disponibilidad, y los mapas secundarios representaron relaciones de uno a muchos sin duplicar información. La reportería confirmó que las estructuras se pueden inspeccionar de manera clara mediante totales, catálogo, índices de autores y préstamos. Las cuatro pruebas automatizadas finalizaron satisfactoriamente, mientras que el benchmark evidenció que las operaciones basadas en hashing resultan más adecuadas que una búsqueda lineal cuando el número de registros crece. Finalmente, se identificó que la solución es apropiada como prototipo educativo, pero debería incorporar persistencia en archivos o una base de datos, validaciones más completas, autenticación y control de copias físicas para convertirse en un sistema de producción. El análisis también mostró que las ventajas de velocidad deben equilibrarse con el consumo de memoria, la ausencia de orden directo y la posibilidad de colisiones en las tablas hash.
Bibliografía	1. Python Software Foundation. (2026). Data structures. Python Documentation. https://docs.python.org/3/tutorial/datastructures.html 2. Tapia-Fernández, S., García-García, D., & García-Hernández, P. (2022). Key concepts, weakness and benchmark on hash table data structures. Algorithms, 15(3), 100.

	https://doi.org/10.3390/a15030100
Anexos	<i>Anexo A. Evidencia de ejecución</i> La siguiente captura resume el reporte de la demostración, la consulta por autor y el benchmark ejecutado.  Salida de la aplicación Biblioteca Digital. <i>Anexo B. Archivos del proyecto</i> El repositorio local entregable contiene biblioteca_app.py, test_biblioteca.py, benchmark.py, grafico_benchmark.py, generar_evidencia.py, el archivo README.md, los resultados de ejecución y el gráfico. Para ejecutar la aplicación se <pre>python3 biblioteca_app.py</pre> utiliza: Para ejecutar las pruebas: <pre>python3 -m unittest -v test_biblioteca.py</pre> El enlace al repositorio GitHub debe agregarse después de publicar la carpeta del proyecto en la cuenta del estudiante. Como respaldo verificable se entrega la carpeta del proyecto junto con este informe, el código fuente y las evidencias generadas. <i>Anexo C. Consideraciones de reproducibilidad</i> Las mediciones no deben interpretarse como valores universales: dependen del hardware, la versión de Python y la carga del sistema. Para reproducirlas, se ejecuta <code>python3 benchmark.py</code>; el script genera evidencias/benchmark.csv. La salida de la demostración se genera con <code>printf</code> <pre>'7\n0\n' python3 biblioteca_app.py.</pre>

